
Customer Churn Prediction

Project Report

Prepared by: Ruwan Pathirana Sanduni Nisansala

Internship ID: CV/A1/45373

Tools Used: Python, pandas, scikit-learn, matplotlib, seaborn

Project: Data Analysis & Machine Learning using Python

1. Introduction

Customer churn is a major concern for different companies, especially for telecommunication companies as it directly impacts revenue and growth. The goal of this project is to analyze customer data, identify factors that influence churn, and build predictive models to accurately classify whether a customer will churn (leave the service) or not.

This project applies data preprocessing, feature encoding, model building, and hyperparameter tuning techniques using Python and machine learning algorithms.

2. Objectives

- To clean and preprocess the telecom churn dataset.
- To identify irrelevant and important features.
- To build predictive models for customer churn classification.
- To evaluate and compare model performance.
- To optimize the best-performing model using hyperparameter tuning.

3. Dataset Overview

- The dataset used is churn-bigml-20.csv, which contains 667 records and 20 features.

Key Columns:

Column	Description
State	Customer's state (categorical)
Account length	Duration of customer's account
International plan	Whether the customer has an international calling plan
Voice mail plan	Whether the customer has a voicemail plan
Total day/eve/night minutes & calls	Call usage during different times
Total intl minutes/calls	International call statistics
Customer service calls	Number of times a customer contacted support
Churn	Target variable indicating if the customer left the company

4. Data Preprocessing

Steps Taken:

1. Data Cleaning:

- ✓ Checked for missing and duplicate values.
- ✓ No missing or duplicate records were found.

2. Dropping Irrelevant Columns:

Columns like 'State' and 'Area code' were dropped as they do not contribute to predicting churn effectively.

3. Encoding Categorical Variables:

Label Encoding was used to convert categorical columns ('International plan', 'Voice mail plan', 'Churn') into numeric format for machine learning compatibility.

4. Feature Scaling:

StandardScaler was used to normalize feature values, ensuring uniform scale across all predictors.

5. Model Building

Three machine learning models were trained and compared:

1. **Logistic Regression** – a statistical model used for binary classification.
2. **Decision Tree Classifier** – a tree-based model that splits data based on feature importance.
3. **Random Forest Classifier** – an ensemble model that combines multiple decision trees for better accuracy and robustness.

6. Model Evaluation

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	92.5%	77.8%	46.7%	58.3%
Decision Tree	88.0%	47.4%	60.0%	52.9%
Random Forest	96.3%	91.7%	73.3%	81.5%

- **Best Model:** Random Forest Classifier

The Random Forest model showed the highest accuracy and overall performance metrics.

7. Confusion Matrix

The confusion matrix helps visualize correct and incorrect predictions:

	Predicted: No Churn	Predicted: Churn
Actual: No Churn	118	1
Actual: Churn	4	11

The Random Forest model correctly classified most churn and non-churn customers.

8. Hyperparameter Tuning (GridSearchCV)

- To improve performance, GridSearchCV was used to find the best combination of model parameters.

Parameter Grid Used:

```
param_grid = {  
    'n_estimators': [100, 200, 300],  
    'max_depth': [None, 10, 20, 30],  
    'min_samples_split': [2, 5, 10],  
    'min_samples_leaf': [1, 2, 4]  
}
```

Explanation:

- **n_estimators:** Number of trees in the forest
- **max_depth:** Maximum depth of trees
- **min_samples_split:** Minimum samples required to split a node
- **min_samples_leaf:** Minimum samples required at each leaf node

GridSearchCV tested different combinations and selected the best parameters based on accuracy using 3-fold cross-validation.

Best Hyperparameters Found:

```
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 300}
```

Best Cross-Validation Accuracy: 90.8%

9. Final Model Evaluation

After tuning, the optimized Random Forest model achieved:

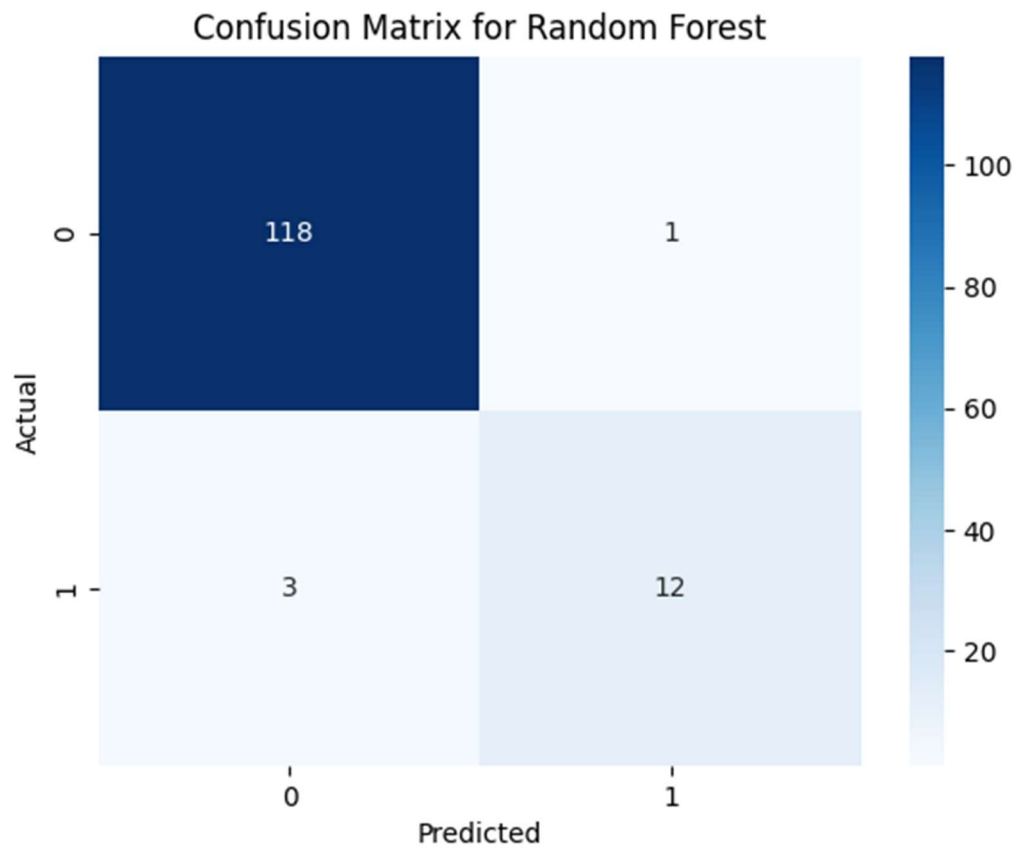
Metric	Value
Accuracy	97.0%
Precision	92.0%
Recall	80.0%
F1 Score	86.0%

The tuned model performed slightly better than the initial version, indicating successful parameter optimization.

10. Visualizations

- **Confusion Matrix Heatmap:**

A heatmap visualization was created to clearly represent the prediction outcomes of the Random Forest model.



```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')  
plt.title('Confusion Matrix for Random Forest')  
plt.xlabel('Predicted')  
plt.ylabel('Actual')  
plt.show()
```

11. Conclusion

- The dataset was clean, well-structured, and required minimal preprocessing.
- The Random Forest Classifier achieved the best performance among tested models.
- Hyperparameter tuning further improved accuracy to 97%.
- The model is reliable and can be used for predicting customer churn with high accuracy.